TimeDistill: Efficient Long-Term Time Series Forecasting with MLP via Cross-Architecture Distillation

Juntong Ni
Emory University
Atlanta, United States
juntong.ni@emory.edu

Zewen Liu
Emory University
Atlanta, United States
zewen.liu@emory.edu

Shiyu Wang*
Independent Researcher
N/A
kwuking@gmail.com

Ming Jin
Griffith University
Brisbane, Australia
mingjinedu@gmail.com

Wei Jin
Emory University
Atlanta, United States
wei.jin@emory.edu

Abstract

Transformer-based and CNN-based methods demonstrate strong performance in long-term time series forecasting. However, their high computational and storage requirements can hinder large-scale deployment. To address this limitation, we propose integrating lightweight MLP with advanced architectures using knowledge distillation (*KD*). Our preliminary study reveals different models can capture complementary patterns, particularly multi-scale and multi-period patterns in the temporal and frequency domains. Based on this observation, we introduce TimeDistill, a cross-architecture *KD* framework that transfers these patterns from teacher models (e.g., Transformers, CNNs) to MLP. Additionally, we provide a theoretical analysis, demonstrating that our *KD* approach can be interpreted as a specialized form of *mixup* data augmentation. TimeDistill improves MLP performance by up to 18.6%, surpassing teacher models on eight datasets. It also achieves up to 7× faster inference and requires 130× fewer parameters. Furthermore, we conduct extensive evaluations to highlight the versatility and effectiveness of TimeDistill. The code is available at Github Code Repo.

CCS Concepts

• **Information systems** → **Temporal data**; • **Mathematics of computing** → **Time series analysis**; • **Computing methodologies** → **Neural networks**.

Keywords

Time Series Forecasting, Knowledge Distillation

ACM Reference Format:
Juntong Ni, Zewen Liu, Shiyu Wang, Ming Jin, and Wei Jin. 2026. TimeDistill: Efficient Long-Term Time Series Forecasting with MLP via Cross-Architecture Distillation. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780218

* Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780218

1 Introduction

Forecasting is a notably critical problem in the time series analysis community, which aims to predict future time series based on historical time series records [45]. It has broad practical applications such as climate modeling [49], traffic flow management [53], healthcare monitoring [16] and finance analytics [10].

Recently, there has been an ongoing debate over which deep learning architecture best suits time series forecasting. The rise of Transformers in various domains [7, 17, 41] has led to their wide adoption in time series forecasting [22, 29, 40, 46, 48, 58, 59], leveraging the strong capabilities of capturing pairwise dependencies and extracting multi-level representations within sequential data. Similarly, CNN architectures have also proven effective by developing convolution blocks for time series [26, 37, 39]. However, despite the strong performance of Transformer-based and CNN-based models, they face significant challenges in large-scale industrial applications due to their relatively high computational demands, especially in latency-sensitive scenarios like financial prediction and healthcare monitoring [10, 16]. In contrast, simpler linear or MLP models offer greater efficiency, although with lower performance [21, 38, 54]. These contrasting observations raises an intriguing question:

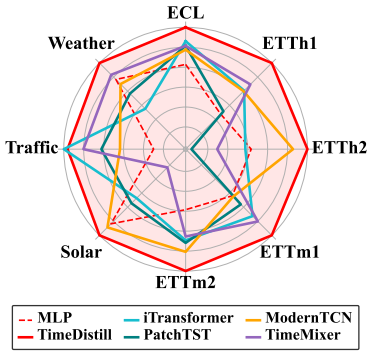


Figure 1: Performance comparison.

> ***Can we combine MLP with other advanced architectures (e.g., Transformers and CNNs) to create a powerful yet efficient model?***

A promising approach to addressing this question is knowledge distillation (*KD*) [13], a technique that transfers knowledge from a larger and more complex model (*teacher*) to a smaller and simpler one (*student*) while maintaining comparable performance. In this work, we pioneer *cross-architecture KD* in time series forecasting,

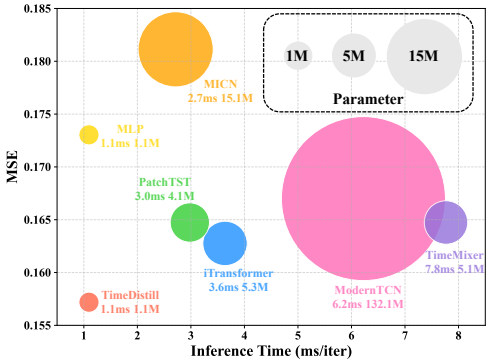


Figure 2: Model efficiency comparison averaged across all prediction lengths (96, 192, 336, 720) for the ECL dataset.

with MLP as the *student* and other advanced architectures (e.g., Transformers and CNNs) as the *teacher*. However, designing such a framework is non-trivial, as it remains unclear what "knowledge" should be distilled into MLP.

To investigate this potential, we conduct a comparative analysis of prediction patterns between MLP and other time series models. Our findings reveal that MLP still excels on some data subsets despite its overall lower performance (Sec. 3.1), which highlights the value of harnessing the *complementary capabilities* across different architectures. To further explore the specific properties to distill, we focus on two key time series patterns: multi-scale pattern in temporal domain and multi-period pattern in frequency domain, given that they are vital in capturing the complex structures typical of many time series. **(1) Multi-Scale Pattern:** Real-world time series often show variations at multiple temporal scales. For example, hourly recorded traffic flow data capture changes within each day, while daily sampled data lose fine-grained details but reveal patterns related to holidays [43]. We observe that models that perform well on the finest scale also perform accurately on coarser scales, while MLP fails on most scales (Sec. 3.2). **(2) Multi-Period Pattern:** Time series often exhibit multiple periodicities. For instance, weather measurements may have both daily and yearly cycles, while electricity consumption data may show weekly and quarterly cycles [47]. We find that models that perform well can capture periodicities similar to those in the ground truth, but MLP fails to capture these periodicities (Sec. 3.2). Therefore, enhancing MLP requires distilling and integrating these multi-scale and multi-period patterns from teacher models.

Based on our observations, we propose a cross-architecture *KD* framework named TimeDistill to bridge the performance and efficiency gap between complex teacher models and a simple MLP. Instead of solely matching predictions in conventional *KD*, TimeDistill focuses on aligning multi-scales and multi-period patterns between MLP and the teacher: we downsample the time series for temporal multi-scale alignment and apply Fast Fourier Transform (FFT) to align period distributions in the frequency domain. The *KD* process can be conducted offline, shifting heavy computations from latency-critical *inference phase*, where millisecond matter, to less time-sensitive *training phase*, where longer processing time is acceptable. We validate effectiveness of TimeDistill both theoretically and empirically and summarize our contributions as follows:

(a) We present *the first cross-architecture KD* framework TimeDistill tailored for efficient and effective time series forecasting via MLP, which is supported by our preliminary studies examining multi-scale and multi-period patterns in time series.
(b) We provide theoretical insights into benefits of TimeDistill, illustrating that proposed distillation process can be viewed as a form of data augmentation through a special *mixup* strategy.
(c) We show that TimeDistill is both effective and efficient, consistently outperforming standalone MLP by up to **18.6%** and surpassing teacher models in nearly all cases (see Figure 1). Additionally, it achieves up to **7x** faster inference and requires up to **130×** fewer parameters compared to teachers (see Figure 2).
(d) We conduct deeper analyses of TimeDistill, exploring its adaptability across various teacher/student models and highlighting the distillation impacts it brings to the temporal and frequency domains.

2 Related Work

2.1 Debate in Long-Term Time Series Forecast

The trade-off between performance and efficiency has prompted a long-standing debate between Transformer-based models and MLP in long-term time series forecasting [21, 44, 54, 57]. Informer [58], Autoformer [48], and FEDformer [59] were among the leading Transformer-based methods. However, recent findings show that a simple Linear or MLP model can achieve performance comparable to these complex Transformer models across various benchmarks while offering significantly better efficiency [54]. This outcome has raised questions about the necessity of Transformers in time series forecasting. Following this, research has moved in two directions. One direction suggests that the issues with Transformer-based models arise from the way they are applied. For example, PatchTST [29] uses patching to preserve local information, and iTransformer [22] focuses on capturing multivariate correlations. These approaches surpass the simple one-layer MLP and demonstrate that Transformers can still deliver strong results in time series forecasting if applied effectively. Meanwhile, CNN-based models have also shown strong performance similar to Transformer-based models. TimesNet [47] transforms 1D time series into 2D variations and applies 2D CNN kernels, MICN [37] adopt multi-scale convolution structures to capture local features and global correlations, and ModernTCN [26] proposes a framework with much larger receptive fields than prior CNN-based structures. Nevertheless, these powerful CNN-based models also face efficiency issues, which further broadens the scope of the debate between performance and efficiency. The other direction focuses on developing lightweight MLP, such as N-BEATS [30], N-hits [3], LightTS [57], FreTS [52], TSMixer [5], TiDE [6], which offer improved efficiency. However, these models typically only match, rather than surpass, the performance of state-of-the-art Transformer-based methods and CNN-based methods. In summary, while Transformer-based and CNN-based models generally offer better performance, simple MLP is more efficient. Therefore, we managed to combine the performance of Transformer-based and CNN-based models with MLP to produce a powerful and efficient model by cross-architecture *KD*.

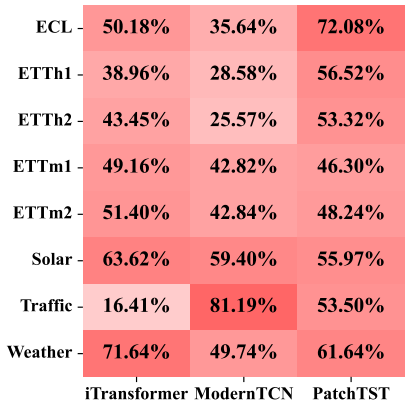


Figure 3: Win ratio (%) of MLP v.s. teacher models across datasets under input-720-predict-96 setting. The win ratio is generally large (average: 49.92%, median: 49.96%), indicating MLP and teacher models excel on different samples with minimal overlap.

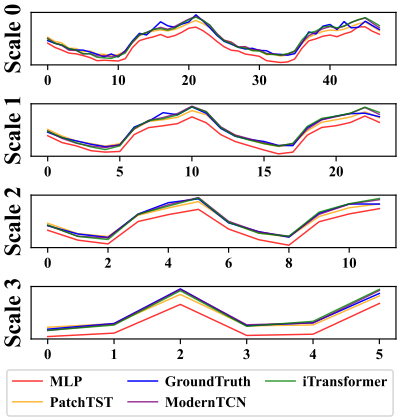


Figure 4: Visualization of model predictions on different downsampled scales of ECL dataset. MLP consistently shows poor performance at multiple scales, while other models perform well, highlighting the importance of capturing multi-scale patterns.

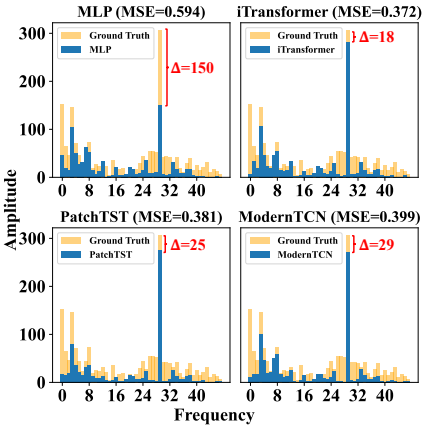


Figure 5: Prediction spectrograms of various models on ECL dataset against the ground truth. MLP fails to match the amplitudes of several main frequencies in the ground truth, with red numbers indicating amplitude differences for the most significant frequency.

2.2 Knolwedge Distillation on Time Series

Knowledge distillation (*KD*) [13, 27] transfers knowledge from a larger, more complex model (teacher) to a smaller, simpler model (student) while maintaining comparable performance. By aligning the output distributions of teacher and student models, *KD* provides richer training signals than hard labels alone, enabling the student to capture subtle patterns that the teacher has learned. In the context of time series, CAKD [50] introduces a two-stage distillation scheme that distills features using adversarial and contrastive learning and performs prediction-level distillation. LightTS [2] designs a *KD* framework specifically for cases where the teacher is an ensemble classifier in time series classification tasks, which limits its applicability to teachers with other architectures. Both of these works do not incorporate time series-specific designs. In contrast, our framework emphasizes extracting essential time series patterns, including multi-scale and multi-period patterns, enabling more effective knowledge distillation. Additionally, we are the first to explore cross-architecture *KD*.

3 Preliminary Studies

In this section, we explore the reasons behind adopting distillation (***What motivates distillation?***) and investigate the specific time series information to distill into the MLP model (***What should we distill?***). We first introduce key notations. For multivariate long-term time series forecasting, given an input time series $\mathbf{X} \in \mathbb{R}^{T \times C}$, where T represents the length of the look-back window and C represents the number of variables, the goal is to predict the future S time steps $\mathbf{Y} \in \mathbb{R}^{S \times C}$.

3.1 What Motivates Distillation?

While MLP excel in efficiency, they often lag in performance compared to Transformer and CNN models or achieve only similar results (Figure 1). However, even if MLP underperforms its teachers overall, it may excel on specific samples. To investigate this, we compare prediction errors of MLP's prediction $\hat{\mathbf{Y}}_s$ and teacher model's prediction $\hat{\mathbf{Y}}_t$ for N samples, defined as $e_s = \{\text{MSE}(\hat{\mathbf{Y}}_s^1, \mathbf{Y}^1), \cdots, \text{MSE}(\hat{\mathbf{Y}}_s^N, \mathbf{Y}^N)\}$ and $e_t = \{\text{MSE}(\hat{\mathbf{Y}}_t^1, \mathbf{Y}^1), \cdots, \text{MSE}(\hat{\mathbf{Y}}_t^N, \mathbf{Y}^N)\}$. We calculate *win ratio*, indicating where MLP outperforms teachers:

$$\text{Win Ratio} = \sum \mathbb{1}(e_s < e_t)/N, \tag{1}$$

where $\mathbb{1}(\cdot)$ equals 1 if MLP outperforms the teacher, otherwise 0. As shown in Figure 3, the win ratio is high (average: 49.92%, median: 49.96%), despite MLP underperforming teacher models overall. For example, on the Traffic dataset, MLP lags behind ModernTCN but wins on 81.19% of the samples, indicating differing strengths on distinct subsets. **This highlights the potential of distilling complementary knowledge from the teacher model into MLP.**

An intuitive strategy of *KD* for MLP is to align predictions with teachers, but this faces limitations. **First**, it risks overfitting noise in the teacher's predictions, leading to less stable knowledge [4, 9, 35]. **Second**, MLP may struggle to replicate the complex patterns such as seasonality, trends, and periodicity in teacher predictions directly due to their limited capacity. **Third**, this approach overlooks valuable knowledge from intermediate features of teacher model. Therefore, the specific knowledge to distill into MLP requires further exploration.

3.2 What Should We Distill?

To investigate which complementary time series patterns to distill into MLP, we analyze prediction patterns in both temporal and frequency domains, considering real-world variations across temporal scales [43] and periodicities [47]. Thus, we conduct further analysis of these two patterns by presenting illustrative cases. **(1) Multi-Scale Pattern:** By downsampling predictions $\hat{\mathbf{Y}}$ using convolutional operations, we obtain their multi-scale representations. Figure 4 shows the analysis of multi-scale temporal patterns by downsampling the time series (*Scale 0*) to coarser scales (*Scales 1–3*).

Models excelling at *Scale 0* generally perform well across all scales. The teacher models align closely with the ground truth at *Scale 3*, capturing underlying trends effectively. In contrast, MLP significantly deviates, showing its limitations in handling multi-scale patterns. **(2) Multi-Period Pattern:** Periodicities in time series are visible in the frequency domain by transforming time series into spectrograms, with the x-axis representing frequency and periodicity calculated as the time series length S divided by frequency. Figure 5 shows spectrograms of model predictions and the ground truth. Models with lower MSE capture periodicities more accurately, closely matching the ground truth at dominant frequencies. In contrast, MLP predictions show larger discrepancies, highlighting its inability to effectively capture periodic patterns.

From our observations, we conjecture that MLP underperforms on certain samples due to their inability to capture essential multi-scale and multi-period patterns, which are essential for effective time series forecasting. To improve MLP, it is crucial to incorporate these complementary patterns from teachers during distillation.

4 Methodology

Motivated by our preliminary studies, we propose a novel *KD* framework TimeDistill for time series to transfer the knowledge from a fixed, pretrained teacher model f_t to a student MLP model f_s. The student produces predictions $\hat{\mathbf{Y}}_s \in \mathbb{R}^{S\times C}$ and internal features $\mathbf{H}_s \in \mathbb{R}^{D\times C}$. The teacher model produces predictions $\hat{\mathbf{Y}}_t \in \mathbb{R}^{S\times C}$ and internal features $\mathbf{H}_t \in \mathbb{R}^{D_t\times C}$. Our general objective is:

$$\min_{\theta_s} \mathcal{L}_{sup}(\mathbf{Y}, \hat{\mathbf{Y}}_s) + \mathcal{L}_{\text{KD}}^{\mathbf{Y}}(\hat{\mathbf{Y}}_t, \hat{\mathbf{Y}}_s) + \mathcal{L}_{\text{KD}}^{\mathbf{H}}(\mathbf{H}_t, \mathbf{H}_s), \tag{2}$$

where θ_s is the parameter of the student; $\mathcal{L}_{sup}$ is the supervised loss (e.g., MSE) between predictions and ground truth; $\mathcal{L}_{\text{KD}}^{\mathbf{Y}}$ and $\mathcal{L}_{\text{KD}}^{\mathbf{H}}$ are the distillation loss terms that encourage student model to learn knowledge from teacher on both **prediction level** [13] and **feature level** [32]. Unlike conventional approaches that emphasize matching model predictions, TimeDistill integrates key time-series patterns including multi-scale and multi-period knowledge. The overall framework of TimeDistill is shown in Figure 6.

4.1 Multi-Scale Distillation

One key component of TimeDistill is multi-scale distillation, where "multi-scale" refers to representing the same time series at different sampling rates. This enables MLP to effectively capture both coarse-grained and fine-grained patterns. By distilling at both the prediction level and the feature level, we ensure that MLP not only replicates the teacher's multi-scale predictions but also aligns with its internal representations from the intermediate layer.

Prediction Level. At the prediction level, we directly transfer multi-scale signals from the teacher's outputs to guide the MLP's predictions. We first produce multi-scale predictions by downsampling the original predictions from the teacher $\hat{\mathbf{Y}}_t \in \mathbb{R}^{S\times C}$ and the MLP $\hat{\mathbf{Y}}_s \in \mathbb{R}^{S\times C}$, where S is the prediction length and C is the number of variables. The predictions at *Scale 0* are equal to the original predictions, that is, $\hat{\mathbf{Y}}_t^0 = \hat{\mathbf{Y}}_t$ and $\hat{\mathbf{Y}}_s^0 = \hat{\mathbf{Y}}_s$. We then downsample these predictions across M scales using convolutional operations with a stride of 2, generating multi-scale prediction sets $\mathcal{Y}_t = \{\hat{\mathbf{Y}}_t^0, \hat{\mathbf{Y}}_t^1, \cdots, \hat{\mathbf{Y}}_t^M\}$ and $\mathcal{Y}_s = \{\hat{\mathbf{Y}}_s^0, \hat{\mathbf{Y}}_s^1, \cdots, \hat{\mathbf{Y}}_s^M\}$, where $\hat{\mathbf{Y}}_t^M, \hat{\mathbf{Y}}_s^M \in \mathbb{R}^{\lfloor S/2^M \rfloor \times C}$. The downsampling is defined as:

$$\hat{\mathbf{Y}}_x^m = \text{Conv}(\hat{\mathbf{Y}}_x^{m-1}, \text{stride} = 2), \tag{3}$$

where $x \in \{t, s\}$, $m \in \{1, \cdots, M\}$, Conv denotes a 1D-convolutional layer with a temporal stride of 2. The predictions at the lowest level $\hat{\mathbf{Y}}_x^0 = \hat{\mathbf{Y}}_x$ maintain the original temporal resolution, while the highest-level predictions $\hat{\mathbf{Y}}_x^M$ represent coarser patterns. We define the multi-scale distillation loss at the prediction level as:

$$\mathcal{L}_{scale}^{\mathbf{Y}} = \sum_{m=0}^{M} ||\hat{\mathbf{Y}}_t^m - \hat{\mathbf{Y}}_s^m||^2/(M+1). \tag{4}$$

Here we use MSE loss to match the MLP's predictions to the teacher's predictions at multiple scales.

Feature Level. At the feature level, we align MLP's intermediate features with teacher's multi-scale representations, enabling MLP to form richer internal structures that support more accurate forecasts. Let $\mathbf{H}_s \in \mathbb{R}^{D\times C}$ and $\mathbf{H}_t \in \mathbb{R}^{D_t\times C}$ denote MLP and teacher features with feature dimensions D and D_t, respectively. As their dimensions can be different, we first use a parameterized regressor (e.g. MLP) to align their feature dimensions:

$$\mathbf{H}_t' = \text{Regressor}(\mathbf{H}_t), \tag{5}$$

where $\mathbf{H}_t' \in \mathbb{R}^{D\times C}$. Similar to the prediction level, we compute $\mathbf{H}_x^m$ by downsampling $\mathbf{H}_s$ and $\mathbf{H}_t'$ across multiple scales using the same approach as in Equation 3. We define the multi-scale distillation loss at the feature level as:

$$\mathcal{L}_{scale}^{\mathbf{H}} = \sum_{m=0}^{M} ||\mathbf{H}_t^m - \mathbf{H}_s^m||^2/(M+1). \tag{6}$$

4.2 Multi-Period Distillation

In addition to multi-scale distillation in the temporal domain, we further propose multi-period distillation to help MLP learn complex periodic patterns in the frequency domain. By aligning periodicity-related signals from the teacher model at both the prediction and feature levels, the MLP can learn richer frequency-domain representations and ultimately improve its forecasting performance.

Prediction Level. For the predictions from the teacher $\hat{\mathbf{Y}}_t \in \mathbb{R}^{S\times C}$ and the MLP $\hat{\mathbf{Y}}_s \in \mathbb{R}^{S\times C}$, we first identify their periodic patterns. We perform this in the frequency domain using the Fast Fourier Transform (FFT):

$$\mathbf{A}_x = \text{Amp}(\text{FFT}(\hat{\mathbf{Y}}_x)), \tag{7}$$

where $x \in \{t, s\}$ and spectrograms $\mathbf{A}_x \in \mathbb{R}^{\frac{S}{2}\times C}$. Here, FFT$(\cdot)$ denotes the FFT operation and Amp$(\cdot)$ calculates the amplitude. We remove the direct current (DC) component from $\mathbf{A}_x$. For certain variable c, the i-th value $\mathbf{A}_x^{i,c}$ indicates the intensity of the frequency-i component, corresponding to a period length $\lceil S/i \rceil$. Larger amplitude values indicate that the associated frequency (period) is more significant.

To reduce the influence of minor frequencies and avoid noise introduced by less meaningful frequencies [47, 59], we propose a distribution-based matching scheme. We use softmax function with a colder temperature to highlight the most significant frequencies:

$$\mathbf{Q}_x^{\mathbf{Y}} = \exp(\mathbf{A}_x^i/\tau)/\sum_{j=1}^{S/2} \exp(\mathbf{A}_x^j/\tau), \tag{8}$$

where $\mathbf{Q}_x^{\mathbf{Y}} \in \mathbb{R}^{\frac{S}{2}\times C}$ and τ is a temperature parameter that controls the sharpness of the distribution. We set $\tau = 0.5$ by default. The period distribution $\mathbf{Q}_x^{\mathbf{Y}}$ represents the multi-period pattern in the

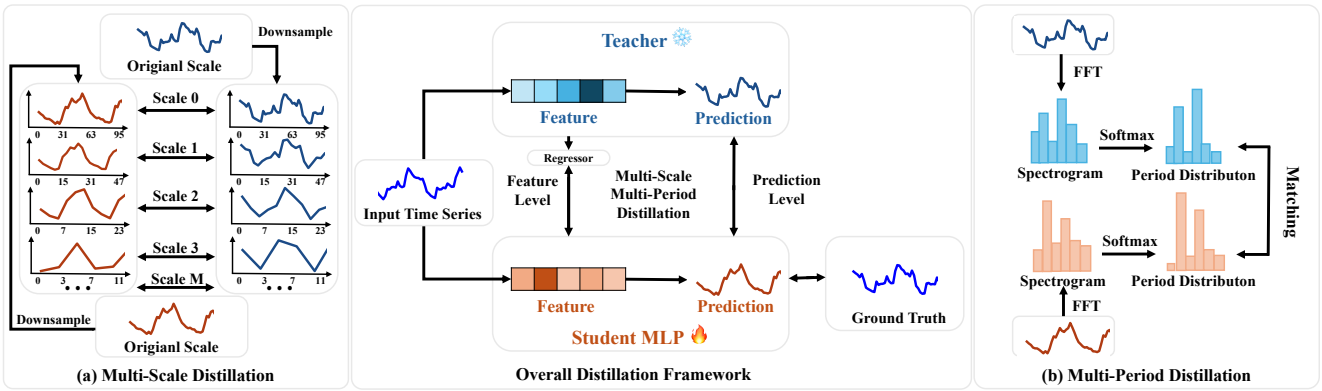


Figure 6: Overall framework of TimeDistill, which distills knowledge from a teacher model to a student MLP using (a) Multi-Scale Distillation and (b) Multi-Period Distillation at both feature and prediction levels. (a) Multi-Scale Distillation involves downsampling the original time series into multiple coarser scales and aligning these scales between the student and teacher. (b) Multi-Period Distillation applies FFT to transform the time series into a spectrogram, followed by matching the period distributions after applying softmax.

prediction time series, which we want the MLP to learn from the teacher. We use KL divergence to match these distributions [13]. We define multi-period distillation loss at the prediction level as:

$$\mathcal{L}^{\mathrm{Y}}_{period} = \mathrm{KL}\big(\mathbf{Q}^{\mathrm{Y}}_t, \mathbf{Q}^{\mathrm{Y}}_s\big). \tag{9}$$

Feature Level. Similar to the prediction level, we apply multi-period distillation at the feature level. For the features $\mathbf{H}'_t \in \mathbb{R}^{D\times C}$ and $\mathbf{H}_s \in \mathbb{R}^{D\times C}$, we compute the spectrograms and the corresponding period distributions $\mathbf{Q}^{\mathrm{H}}_x$ using the same approach as in Equations 7 and 8. Multi-period distillation loss at feature level is then defined as:

$$\mathcal{L}^{\mathrm{H}}_{period} = \mathrm{KL}\big(\mathbf{Q}^{\mathrm{H}}_t, \mathbf{Q}^{\mathrm{H}}_s\big). \tag{10}$$

4.3 Overall Optimization and Theoretical Analysis

During the training of TimeDistill, we jointly optimize both the multi-scale and multi-period distillation losses at both the prediction and feature levels, together with the supervised ground-truth label loss:

$$\mathcal{L}_{sup} = ||\mathrm{Y} - \hat{\mathrm{Y}}_s||^2, \tag{11}$$

where $\mathcal{L}_{sup}$ is the ground-truth loss (for example, MSE loss) used to train MLP directly. Thus, the overall training loss for the student MLP is defined as:

$$\mathcal{L} = \mathcal{L}_{sup} + \alpha \cdot \big(\mathcal{L}^{\mathrm{Y}}_{scale} + \mathcal{L}^{\mathrm{Y}}_{period}\big) + \beta \cdot \big(\mathcal{L}^{\mathrm{H}}_{scale} + \mathcal{L}^{\mathrm{H}}_{period}\big), \tag{12}$$

where α and β are hyper-parameters that control the contributions of the prediction-level and feature-level distillation loss terms, respectively. The teacher model is pretrained and remains frozen throughout the training process of MLP.

Theoretical Interpretations. We provide a theoretical understanding of multi-scale and multi-period distillation loss from **a novel data augmentation perspective**. We further show that the proposed distillation loss can be interpreted as training with augmented samples derived from a special *mixup* [55] strategy. The distillation process augments data by blending ground truth with teacher predictions, analogous to label smoothing in classification, and provides several benefits for time series forecasting: ***(1)*** *Enhanced Generalization:* It enhances generalization by exposing the student model to richer supervision signals from augmented samples, thus mitigating overfitting, especially with limited or noisy data. ***(2)*** *Explicit Integration of Patterns:* The augmented supervision signals explicitly incorporate patterns across multiple scales and periods, offering insights that are not immediately evident in the raw ground truth. ***(3)*** *Stabilized Training Dynamics:* The blending of targets softens the supervision signals, which diminishes the model's sensitivity to noise and leads to more stable training phases. This will in turn support smoother optimization dynamics and fosters improved convergence. For clarity, our discussion is centered at the prediction level. We present the following theorem:

Theorem 4.1. *Let (x, y) denote original input data pairs and (x, y^t) represent corresponding teacher data pairs. Consider a data augmentation function $\mathcal{A}(\cdot)$ applied to (x, y), generating augmented samples (x', y'). Define the training loss on these augmented samples as $\mathcal{L}_{aug} = \sum_{(x',y')\in\mathcal{A}(x,y)} |f_s(x') - y'|^2$. Then, the following inequality holds: $\mathcal{L}_{sup} + \eta\mathcal{L}_{scale} \geq \mathcal{L}_{aug}$, when $\mathcal{A}(\cdot)$ is instantiated as a mixup function [55] that interpolates between the original input data (x, y) and teacher data (x, y^t) with a mixing coefficient $\lambda = \frac{1}{1+\eta}$, i.e. $y' = \lambda y + (1-\lambda)y^t$.*

We provide proof of Theorem 4.1 in Appendix A. Theorem 4.1 suggests that optimizing multi-scale distillation loss $\mathcal{L}_{\text{scale}}$ jointly with supervised loss $\mathcal{L}_{\text{sup}}$ is equivalent to minimizing an upper bound on a special *mixup* augmentation loss. In particular, we mix multi-scale teacher predictions $\{\hat{\mathrm{Y}}^{(m)}_t\}^M_{m=0}$ with ground truth Y, allowing MLP to learn more informative time series temporal pattern. Similarly, we present a theorem for understanding $\mathcal{L}_{period}$.

Theorem 4.2. *Define the training loss on the augmented samples using KL divergence as $\mathcal{L}_{aug} = \sum_{(x',y')\in\mathcal{A}(x,y)} KL\big(y', \mathcal{X}(f_s(x'))\big)$, where $\mathcal{X}(\cdot) = Softmax(FFT(\cdot))$. Then, the following inequality holds: $\mathcal{L}_{sup} + \eta\mathcal{L}_{period} \geq \mathcal{L}_{aug}$, where $\mathcal{A}(\cdot)$ is instantiated as a mixup function that interpolates between the period distribution of original input data $(x, \mathcal{X}(y))$ and teacher data $(x, \mathcal{X}(y^t))$ with a mixing coefficient $\lambda = \eta$, i.e. $y' = \mathcal{X}(y) + \lambda\mathcal{X}(y^t)$.*

The proof can be found in Appendix A. Theorem 4.2 shows that optimizing the multi-period distillation loss $\mathcal{L}_{\text{period}}$ jointly with the supervised loss $\mathcal{L}_{\text{sup}}$ is equivalent to minimizing an upper bound on the KL divergence between the student period distribution $\mathcal{X}(f_s(x'))$ (or $\mathbf{Q}_s$) and a *mixed* period distribution y' (or $\mathbf{Q}_y + \lambda\,\mathbf{Q}_t$).

5 Experiment

5.1 Experimental Setup

Datasets and Baselines. We run experiments to evaluate the performance and efficiency of TimeDistill on 8 widely used benchmarks: Electricity (ECL), the ETT datasets (ETTh1, ETTh2, ETTm1, ETTm2), Solar, Traffic, and Weather, following [22, 26, 43]. To examine the effectiveness of our method across diverse tasks, we compare TimeDistill with 8 baseline models that cover a range of architectures. Specifically, we use *Transformer-based models*: iTransformer [22], PatchTST [29], FEDformer [59], Autoformer [48]; *CNN-based models*: ModernTCN [26], MICN [37], TimesNet [47]; and an *MLPs-based model*: TimeMixer [43]. More details can be found in Appendix B.

5.2 Main Results

Table 1 presents the long-term time series forecasting performance of the proposed TimeDistill compared with previous state-of-the-art baselines. By default, TimeDistill uses ModernTCN as the teacher, though results with alternative teachers are provided in Sec. 5.3. Notably, TimeDistill outperforms the baselines on **7 out of 8** datasets on MSE and **all** datasets on MAE. Furthermore, TimeDistill consistently exceeds the performance of its teacher (ModernTCN) by up to **5.37%** and improves over vanilla MLP by up to **13.87%**, as shown in Table 2. These results highlight the effectiveness of our multi-scale and multi-period distillation approach in transferring knowledge for enhanced forecasting performance.

Efficiency Analysis. Beyond its strong predictive performance, another notable advantage of TimeDistill is its extremely lightweight architecture, as it is simply an MLP. Figure 2 in the Introduction section shows the trade-off between inference time, memory footprint, and performance. We can observe that TimeDistill can achieve up to **7**× speedup and up to **130**× fewer parameters compared with baselines. This property makes TimeDistill suitable for deployment on devices with limited computational resources and in latency-sensitive applications that require fast inference.

5.3 Versatility of TimeDistill

In this subsection, we further demonstrate the versatility of the proposed TimeDistill by evaluating its performance under different configurations, including variations in teacher/student models and look-back window lengths.

Different Teachers. We use ModernTCN as the teacher model for our main results. In Table 2, we present results using other teacher models, such as iTransformer, TimeMixer, and PatchTST. We observe that TimeDistill can effectively learn from various teachers, improving MLP by up to **13.87%**. Furthermore, TimeDistill achieves significant performance improvements over the teachers themselves, with gains of up to **21.41%**. We hypothesize two key reasons for these improvements. First, the student MLP model already demonstrates strong learning capabilities; for instance, on the Solar dataset, MLP outperforms most teachers. Second, the multi-scale and multi-period *KD* approach delivers both temporal and frequency knowledge from teachers to MLP, offering additional valuable insights. Similar findings have been reported in recent *KD* studies [1, 11, 56], which suggest that integrating diverse views from multiple models can enhance performance.

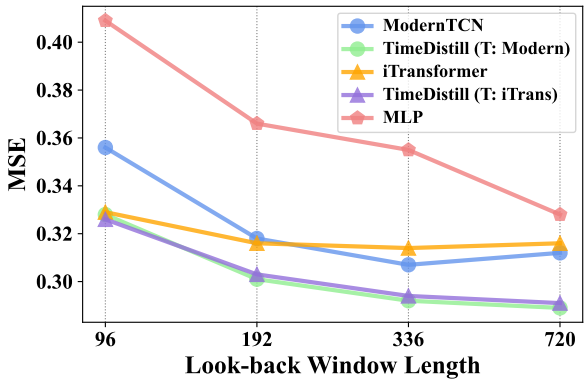


Figure 7: Forecasting performance is evaluated with different look-back lengths and the results are averaged across all prediction lengths $S \in \{96, 192, 336, 720\}$ on eight datasets.

Different Students. To evaluate whether TimeDistill can enhance the performance of other lightweight models, we select two MLPs-based models, TSMixer [5], LightTS [57], as students. Unlike a simple MLP, these MLPs-based models consist of multiple MLPs and operate in a channel-dependent manner. Consequently, they are generally more complex than a standard MLP, which results in reduced efficiency. Besides, we select a Linear-based model FITS [51], which is extremely lightweight. We use ModernTCN as the teacher model, consistent with other experiments. As shown in Table 3, TimeDistill consistently improves the performance of TSMixer, LightTS, and FITS, achieving remarkable MSE reductions of **6.26%**, **8.02%**, and **3.96%**, respectively. These results demonstrate that TimeDistill is highly adaptable and can effectively enhance other lightweight methods.

Different Look-Back Window Lengths. The length of the look-back window significantly influences forecasting accuracy, as it determines how much historical information can be utilized for learning. Figure 7 presents the average MSE results across all eight datasets. Overall, the performance of models in the figure, particularly MLP, generally improves as the look-back window size increases. Notably, TimeDistill consistently enhances the performance of MLP and outperforms the teacher models across all look-back window lengths.

5.4 Deeper Analysis into Our Distillation Framework

Ablation study. As our proposed TimeDistill incorporates two distillation strategies, *multi-scale distillation* and *multi-period distillation*, we assess their effectiveness by removing the corresponding losses, $\mathcal{L}_{\text{scale}}$ and $\mathcal{L}_{\text{period}}$, from TimeDistill. Additionally, we evaluate the impact of *prediction-level* and *feature-level distillation* by removing $\mathcal{L}^{\text{H}}$ and $\mathcal{L}^{\text{Y}}$, respectively. Furthermore, we test the model

Table 1: Long-term time series forecasting results with prediction lengths $S \in \{96, 192, 336, 720\}$. A lower MSE or MAE indicates a better prediction. For consistency, we maintain a fixed input length of 720 throughout all the experiments. Results are averaged from all prediction lengths. The best performance is highlighted in red, and the second-best is underlined.

Models	TimeDistill (Ours)		iTranformer (2024)		ModernTCN (2024)		TimeMixer (2024d)		PatchTST (2023)		MICN (2023a)		FEDformer (2022)		TimesNet (2022)		Autoformer (2021)	
Metric	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
ECL	**0.157**	**0.254**	0.163	0.259	0.167	0.262	0.165	0.259	0.165	0.266	0.181	0.293	0.274	0.376	0.250	0.347	0.238	0.347
ETTh1	**0.429**	**0.441**	0.468	0.476	0.469	0.465	0.459	0.465	0.498	0.490	0.739	0.631	0.527	0.524	0.507	0.500	0.731	0.659
ETTh2	**0.345**	**0.395**	0.398	0.426	0.357	0.403	0.422	0.444	0.444	0.443	1.078	0.736	0.456	0.485	0.419	0.446	1.594	0.940
ETTm1	**0.348**	**0.379**	0.372	0.402	0.390	0.410	0.367	0.388	0.383	0.412	0.439	0.461	0.423	0.451	0.398	0.419	0.570	0.526
ETTm2	**0.244**	**0.311**	0.276	0.337	0.267	0.330	0.279	0.339	0.274	0.335	0.348	0.404	0.359	0.401	0.291	0.349	0.420	0.448
Solar	**0.184**	**0.241**	0.214	0.270	0.191	0.243	0.238	0.288	0.210	0.257	0.213	0.277	0.300	0.383	0.196	0.262	1.037	0.742
Traffic	0.387	**0.271**	**0.379**	0.271	0.413	0.284	0.391	0.275	0.402	0.284	0.500	0.316	0.629	0.388	0.693	0.399	0.696	0.427
Weather	**0.220**	**0.269**	0.259	0.290	0.238	0.277	0.230	0.271	0.246	0.283	0.240	0.292	0.355	0.398	0.257	0.294	0.471	0.465

Table 2: Performance improvement by TimeDistill with different teachers. Δ_{MLP}, $\Delta_{Teacher}$ indicate the improvement of MLP+TimeDistill over a trained MLP and Teacher, respectively. We report average MSE across all prediction lengths.

Teacher Models		iTranformer (2024)	ModernTCN (2024)	TimeMixer (2024d)	PatchTST (2023)
ECL	Teacher	0.163	0.167	0.165	0.165
	MLP	0.173	0.173	0.173	0.173
	+TimeDistill	**0.157**	**0.157**	**0.159**	**0.159**
	$\Delta_{Teacher}$	3.68%	5.61%	3.31%	3.64%
	Δ_{MLP}	9.27%	9.09%	7.94%	8.11%
ETT(avg)	Teacher	0.379	0.371	0.382	0.400
	MLP	0.397	0.397	0.397	0.397
	+TimeDistill	**0.345**	**0.342**	**0.353**	**0.358**
	$\Delta_{Teacher}$	8.92%	7.94%	7.46%	10.38%
	Δ_{MLP}	13.06%	13.87%	10.91%	9.65%
Solar	Teacher	0.214	0.191	0.288	0.210
	MLP	0.194	0.194	0.194	**0.194**
	+TimeDistill	**0.185**	**0.184**	**0.187**	0.204
	$\Delta_{Teacher}$	13.55%	3.60%	21.41%	2.86%
	Δ_{MLP}	4.80%	5.14%	3.58%	-4.98%
Traffic	Teacher	**0.379**	0.413	0.391	0.402
	MLP	0.434	0.434	0.434	0.434
	+TimeDistill	0.389	**0.387**	**0.391**	**0.390**
	$\Delta_{Teacher}$	-2.64%	6.32%	-0.04%	2.99%
	Δ_{MLP}	10.30%	10.70%	9.76%	10.07%
Weather	Teacher	0.259	0.238	0.230	0.246
	MLP	0.234	0.234	0.234	0.234
	+TimeDistill	**0.220**	**0.220**	**0.219**	**0.220**
	$\Delta_{Teacher}$	15.06%	7.37%	4.82%	10.57%
	Δ_{MLP}	5.83%	5.66%	6.16%	5.83%

by removing the supervised loss $\mathcal{L}_{\text{sup}}$, using only the distillation losses as the overall loss for TimeDistill.

Table 4 presents the results of these ablations, compared with the full TimeDistill, a stand-alone MLP, and the teacher models. We draw the following observations: **First**, each loss term at both levels, when used individually, already outperforms the stand-alone MLP. **Second**, when combined, the losses complement each other, enabling TimeDistill to achieve the best performance, surpassing the teacher model. **Third**, notably, TimeDistill maintains superior performance over both the MLP and the teacher even without

Table 3: Performance improvement of TimeDistill with different students on ETTh1, averaged across all prediction lengths. $\Delta_{Student}$ represents the improvement of Student+TimeDistill over the original student.

Student Models	MLP		LightTS		TSMixer		FITS	
	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Student	0.502	0.489	0.465	0.471	0.471	0.474	0.428	0.443
+TimeDistill	**0.428**	**0.445**	**0.436**	**0.445**	**0.433**	**0.446**	**0.411**	**0.430**
$\Delta_{Student}$	14.74%	9.00%	6.26%	5.57%	8.02%	5.92%	3.96%	2.75%

Table 4: Ablation study measured by MSE on different components of TimeDistill (Teacher: ModernTCN). The best performance is in red, and the second-best is underlined.

Method	ECL	ETT(avg)	Solar	Traffic	Weather
Teacher	0.167	0.371	0.191	0.413	0.238
MLP	0.173	0.397	0.194	0.434	0.234
TimeDistill	**0.157**	**0.342**	**0.184**	**0.387**	**0.220**
w/o prediction level	0.157	0.373	0.184	0.392	0.221
w/o feature level	0.161	0.349	0.188	0.393	0.224
w/o multi-scale	0.162	0.377	0.187	0.393	0.224
w/o multi-period	0.157	0.342	0.184	0.391	0.221
w/o sup	0.165	0.344	0.192	0.506	0.225

the supervised loss $\mathcal{L}_{\text{sup}}$. This can be attributed to two potential reasons: (1) ground truth may contain noise, making it more challenging to fit, whereas teacher provides simpler and more learnable knowledge; and (2) *multi-scale* and *multi-period* distillation processes effectively transfer complementary knowledge from teacher to MLP.

Does TimeDistill Truly Enhance MLP's Learning from the Teacher? As shown in Table 5, TimeDistill improves the win ratio of MLP over the teacher by an average of 17.46% across eight datasets. To determine whether this improvement is due to TimeDistill succeeding on samples MLP previously failed, we present the *Win Keep* results. A higher *Win Keep* indicates that TimeDistill's improvements come from previously failed samples. *Win Keep* remains consistently high across datasets (above 76.61%, with an average of 84.55%), indicating that TimeDistill

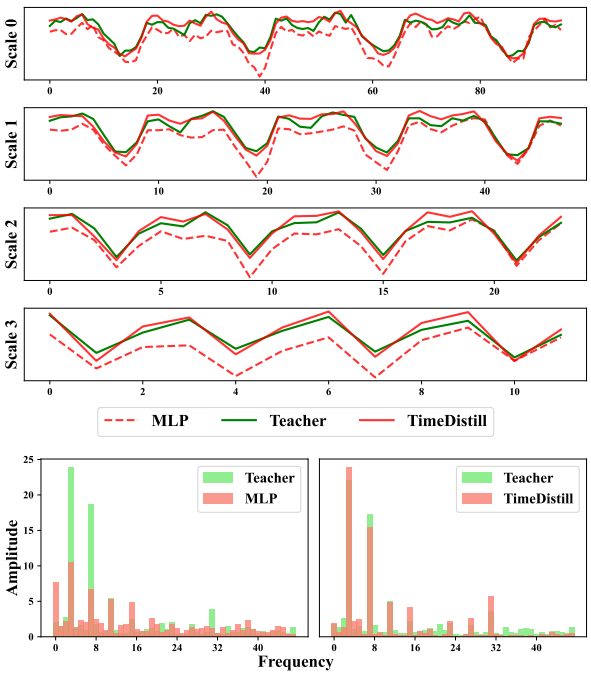


Figure 8: Prediction comparison across temporal scales and spectrograms before and after distillation on ETTh1. MSE for MLP, Teacher (ModernTCN), and TimeDistill are 0.790, 0.365, and 0.366, showing TimeDistill bridges temporal and frequency domain gaps via multi-scale and multi-period distillation.

Table 5: Win ratio (%) of MLP and TimeDistill vs. ModernTCN under input-720-predict-96 setting. U_M and U_T denote samples where MLP and TimeDistill outperform the teacher. *Win Keep*, $\frac{|U_M \cap U_T|}{|U_M|}$, shows TimeDistill's retention of MLP's wins.

Dataset	MLP	TimeDistill	*Win Keep*
ECL	35.64%	59.69%	95.44%
ETTh1	28.58%	58.35%	79.27%
ETTh2	25.57%	57.63%	69.10%
ETTm1	42.82%	63.12%	84.61%
ETTm2	42.84%	57.99%	80.14%
Solar	59.40%	61.02%	91.77%
Traffic	81.19%	91.24%	99.46%
Weather	49.74%	56.43%	76.61%

not only retains MLP's success on samples where it already outperformed the teacher but also allows MLP to outperform the teacher on many samples it previously struggled with. This demonstrates that TimeDistill effectively transfers knowledge from the teacher to student MLP, enabling MLP to perform better on challenging samples while maintaining its existing strengths.

Does TimeDistill Effectively Bridge the Gap Between Student and Teacher? We present a case visualization from the ETTh1 dataset in Figure 8. The figure shows that TimeDistill reduces the difference between the teacher model (ModernTCN) and the student model (MLP) at multiple scales, and makes the fine-level series prediction more precise by transferring trend patterns at coarser scales from the teacher. In addition, for the same case, we visualize the spectrogram of the prediction and find that TimeDistill also helps reduce the difference between the teacher and MLP in the frequency domain, enabling the MLP to learn more structured multi-period patterns.

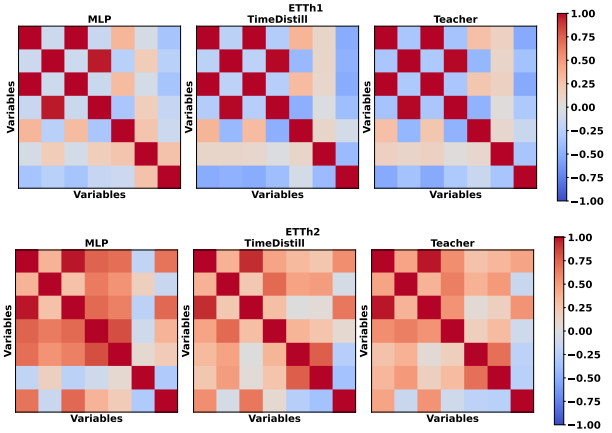


Figure 9: Multi-Variate Person Correlation Visualization on ETTh1 and ETTh2 datasets.

Does TimeDistill Learn Multi-Variate Correlation via Distillation? Some teacher models, such as iTransformer and ModerTCN, are designed in a channel-dependent (CD) manner. While this approach can utilize multi-variate correlation effectively, it incurs a higher computational burden. In contrast, advanced methods like PatchTST, TimeMixer, and our method TimeDistill are employed in a channel-independent (CI) manner, which offers better computational efficiency. This leads to an intriguing question: *Can TimeDistill enable MLP to learn multi-variate correlation from the teacher without an explicit distillation design?* To explore this, we present a preliminary study using Figure 9, which illustrates two cases from the ETTh1 and ETTh2 datasets, each containing seven variables. Our observations reveal that TimeDistill's multi-variate correlation more closely resembles that of the teacher compared to the MLP model before distillation. This suggests that TimeDistill facilitates learning underlying structures among multi-variates through distillation even though the student MLP is a CI model. These findings highlight a potential benefit of distillation that merits further exploration. We leave this for further work.

6 Conclusion and Future work

We propose TimeDistill, a cross-architecture *KD* framework enabling lightweight MLP to surpass complex teachers. By distilling multi-scale and multi-period patterns, TimeDistill efficiently transfers temporal and frequency-domain knowledge to student MLP. Theoretical interpretations and comprehensive experiments confirm its versatility and effectiveness. Future work includes distilling from advanced time series models, e.g. time series foundation models, and incorporating multivariate patterns.

References

[1] Zeyuan Allen-Zhu and Yuanzhi Li. 2020. Towards understanding ensemble, knowledge distillation and self-distillation in deep learning. *arXiv preprint arXiv:2012.09816* (2020).

[2] David Campos, Miao Zhang, Bin Yang, Tung Kieu, Chenjuan Guo, and Christian S Jensen. 2023. LightTS: Lightweight time series classification with adaptive ensemble distillation. *Proceedings of the ACM on Management of Data* 1, 2 (2023), 1–27.

[3] Cristian Challu, Kin G Olivares, Boris N Oreshkin, Federico Garza Ramirez, Max Mergenthaler Canseco, and Artur Dubrawski. 2023. Nhits: Neural hierarchical interpolation for time series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 6989–6997.

[4] Guobin Chen, Wongun Choi, Xiang Yu, Tony Han, and Manmohan Chandraker. 2017. Learning efficient object detection models with knowledge distillation. *Advances in neural information processing systems* 30 (2017).

[5] Si-An Chen, Chun-Liang Li, Nate Yoder, Sercan O Arik, and Tomas Pfister. 2023. Tsmixer: An all-mlp architecture for time series forecasting. *arXiv preprint arXiv:2303.06053* (2023).

[6] Abhimanyu Das, Weihao Kong, Andrew Leach, Shaan Mathur, Rajat Sen, and Rose Yu. 2023. Long-term forecasting with tide: Time-series dense encoder. *arXiv preprint arXiv:2304.08424* (2023).

[7] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *CoRR* abs/1810.04805 (2018). arXiv:1810.04805

[8] Tianning Feng, Juntong Ni, Ezequiel Gleichgerrcht, and Wei Jin. 2025. SeizureFormer: A Transformer Model for IEA-Based Seizure Risk Forecasting. *arXiv preprint arXiv:2504.16098* (2025).

[9] Amit Gajbhiye, Marina Fomicheva, Fernando Alva-Manchego, Frédéric Blain, Abiola Obamuyide, Nikolaos Aletras, and Lucia Specia. 2021. Knowledge distillation for quality estimation. *arXiv preprint arXiv:2107.00411* (2021).

[10] Clive William John Granger and Paul Newbold. 2014. *Forecasting economic time series*. Academic press.

[11] Zhichun Guo, William Shiao, Shichang Zhang, Yozen Liu, Nitesh V Chawla, Neil Shah, and Tong Zhao. 2023. Linkless link prediction via relational distillation. In *International Conference on Machine Learning*. PMLR, 12012–12033.

[12] Lu Han, Han-Jia Ye, and De-Chuan Zhan. 2024. The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting. *IEEE Transactions on Knowledge and Data Engineering* (2024).

[13] Geoffrey Hinton. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531* (2015).

[14] Johan Ludwig William Valdemar Jensen. 1906. Sur les fonctions convexes et les inégalités entre les valeurs moyennes. *Acta mathematica* 30, 1 (1906), 175–193.

[15] Ming Jin, Shiyu Wang, Lintao Ma, Zhixuan Chu, James Y Zhang, Xiaoming Shi, Pin-Yu Chen, Yuxuan Liang, Yuan-Fang Li, Shirui Pan, et al. 2023. Time-llm: Time series forecasting by reprogramming large language models. *arXiv preprint arXiv:2310.01728* (2023).

[16] Shruti Kaushik, Abhinav Choudhury, Pankaj Kumar Sheron, Nataraj Dasgupta, Sayee Natarajan, Larry A Pickett, and Varun Dutt. 2020. AI in healthcare: time-series forecasting using statistical, neural, and ensemble architectures. *Frontiers in big data* 3 (2020), 4.

[17] Salman Khan, Muzammal Naseer, Munawar Hayat, Syed Waqas Zamir, Fahad Shahbaz Khan, and Mubarak Shah. 2022. Transformers in vision: A survey. *ACM computing surveys (CSUR)* 54, 10s (2022), 1–41.

[18] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. 2021. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. In *International Conference on Learning Representations*.

[19] Taehyeon Kim, Jaehoon Oh, NakYil Kim, Sangwook Cho, and Se-Young Yun. 2021. Comparing kullback-leibler divergence and mean squared error loss in knowledge distillation. *arXiv preprint arXiv:2105.08919* (2021).

[20] Diederik P Kingma. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).

[21] Shengsheng Lin, Weiwei Lin, Wentai Wu, Haojun Chen, and Junjie Yang. 2024. SparseTSF: Modeling Long-term Time Series Forecasting with 1k Parameters. *arXiv preprint arXiv:2405.00946* (2024).

[22] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*.

[23] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2022. Non-stationary Transformers: Exploring the Stationarity in Time Series Forecasting. (2022).

[24] Zewen Liu, Juntong Ni, Max SY Lau, and Wei Jin. 2025. Cape: Covariate-adjusted pre-training for epidemic time series forecasting. *arXiv preprint arXiv:2502.03393* (2025).

[25] Zewen Liu, Juntong Ni, Xianfeng Tang, Max SY Lau, Wenpeng Yin, and Wei Jin. 2025. Can Large Language Models Adequately Perform Symbolic Reasoning Over Time Series? *arXiv preprint arXiv:2508.03963* (2025).

[26] Donghao Luo and Xue Wang. 2024. Moderntcn: A modern pure convolution structure for general time series analysis. In *The Twelfth International Conference on Learning Representations*.

[27] Juntong Ni, Saurabh Kataria, Shengpu Tang, Carl Yang, Xiao Hu, and Wei Jin. 2025. Ppg-distill: Efficient photoplethysmography signals analysis via foundation model distillation. *arXiv preprint arXiv:2509.19215* (2025).

[28] Juntong Ni, Shiyu Wang, Zewen Liu, Xiaoming Shi, Xinyue Zhong, Zhou Ye, and Wei Jin. 2025. U-Cast: Learning Hierarchical Structures for High-Dimensional Time Series Forecasting. *arXiv preprint arXiv:2507.15119* (2025).

[29] Yuqi Nie, Nam H Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *The Eleventh International Conference on Learning Representations*.

[30] Boris N. Oreshkin, Dmitri Carpov, Nicolas Chapados, and Yoshua Bengio. 2020. N-BEATS: Neural basis expansion analysis for interpretable time series forecasting. In *International Conference on Learning Representations*.

[31] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[32] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. 2014. Fitnets: Hints for thin deep nets. *arXiv preprint arXiv:1412.6550* (2014).

[33] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. 2024. Scaling to Billion Parameters for Time Series Foundation Models with Mixture of Experts. In *NeurIPS Workshop on Time Series in the Age of Large Models*.

[34] Xiaoming Shi, Shiyu Wang, Yuqi Nie, Dianqi Li, Zhou Ye, Qingsong Wen, and Ming Jin. 2024. Time-moe: Billion-scale time series foundation models with mixture of experts. *arXiv preprint arXiv:2409.16040* (2024).

[35] Makoto Takamoto, Yusuke Morishita, and Hitoshi Imaoka. 2020. An efficient method of training small models for regression problems with knowledge distillation. In *2020 IEEE Conference on Multimedia Information Processing and Retrieval (MIPR)*. IEEE, 67–72.

[36] MTCAJ Thomas and A Thomas Joy. 2006. *Elements of information theory*. Wiley-Interscience.

[37] Huiqiang Wang, Jian Peng, Feihu Huang, Jince Wang, Junhui Chen, and Yifei Xiao. 2023. MICN: Multi-scale Local and Global Context Modeling for Long-term Series Forecasting. In *The Eleventh International Conference on Learning Representations*.

[38] Shiyu Wang. 2024. Neuralreconciler for hierarchical time series forecasting. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining*. 731–739.

[39] Shiyu Wang, Zhixuan Chu, Yinbo Sun, Yu Liu, Yuliang Guo, Yang Chen, Huiyang Jian, Lintao Ma, Xingyu Lu, and Jun Zhou. 2024. Multiscale representation enhanced temporal flow fusion model for long-term workload forecasting. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*. 4948–4956.

[40] Shiyu Wang, Jiawei Li, Xiaoming Shi, Zhou Ye, Baichuan Mo, Wenze Lin, Shengtong Ju, Zhixuan Chu, and Ming Jin. 2024. Timemixer++: A general time series pattern machine for universal predictive analysis. *arXiv preprint arXiv:2410.16032* (2024).

[41] Shiyu Wang, Wei Lu, Jiawei Li, Xiaoming Shi, Xinyue Zhong, Zhou Ye, Ming Jin, and Qingsong Wen. 2025. FRT: Flow-based Reconcile Transformer for Hierarchical Time Series. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 4995–5006.

[42] Shiyu Wang, Yinbo Sun, Yan Wang, Fan Zhou, Lin-Tao Ma, James Zhang, and YangFei Zheng. 2023. Flow-based end-to-end model for hierarchical time series forecasting via trainable attentive-reconciliation. In *International Conference on Database Systems for Advanced Applications*. Springer, 167–176.

[43] Shiyu Wang, Haixu Wu, Xiaoming Shi, Tengge Hu, Huakun Luo, Lintao Ma, James Y Zhang, and Jun Zhou. 2024. Timemixer: Decomposable multiscale mixing for time series forecasting. *arXiv preprint arXiv:2405.14616* (2024).

[44] Shiyu Wang, Fan Zhou, Yinbo Sun, Lintao Ma, James Zhang, and Yangfei Zheng. 2022. End-to-end modeling of hierarchical time series using autoregressive transformer and conditional normalizing flow-based reconciliation. In *2022 IEEE International Conference on Data Mining Workshops (ICDMW)*. IEEE, 1087–1094.

[45] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Yong Liu, Mingsheng Long, and Jianmin Wang. 2024. Deep time series models: A comprehensive survey and benchmark. *arXiv preprint arXiv:2407.13278* (2024).

[46] Qingsong Wen, Tian Zhou, Chaoli Zhang, Weiqi Chen, Ziqing Ma, Junchi Yan, and Liang Sun. 2022. Transformers in time series: A survey. *arXiv preprint arXiv:2202.07125* (2022).

[47] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2022. Timesnet: Temporal 2d-variation modeling for general time series analysis. *arXiv preprint arXiv:2210.02186* (2022).

[48] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *Advances in neural information processing systems* 34 (2021), 22419–22430.

[49] Haixu Wu, Hang Zhou, Mingsheng Long, and Jianmin Wang. 2023. Interpretable weather forecasting for worldwide stations with a unified deep model. *Nature Machine Intelligence* 5, 6 (2023), 602–611.
[50] Qing Xu, Zhenghua Chen, Mohamed Ragab, Chao Wang, Min Wu, and Xiaoli Li. 2022. Contrastive adversarial knowledge distillation for deep model compression in time-series regression tasks. *Neurocomputing* 485 (2022), 242–251.
[51] Zhijian Xu, Ailing Zeng, and Qiang Xu. 2023. FITS: Modeling time series with $10k$ parameters. *arXiv preprint arXiv:2307.03756* (2023).
[52] Kun Yi, Qi Zhang, Wei Fan, Shoujin Wang, Pengyang Wang, Hui He, Ning An, Defu Lian, Longbing Cao, and Zhendong Niu. 2023. Frequency-domain MLPs are More Effective Learners in Time Series Forecasting. In *Thirty-seventh Conference on Neural Information Processing Systems*.
[53] Xueyan Yin, Genze Wu, Jinze Wei, Yanming Shen, Heng Qi, and Baocai Yin. 2021. Deep learning on traffic prediction: Methods, analysis, and future directions. *IEEE Transactions on Intelligent Transportation Systems* 23, 6 (2021), 4927–4943.
[54] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 37. 11121–11128.
[55] Hongyi Zhang. 2017. mixup: Beyond empirical risk minimization. *arXiv preprint arXiv:1710.09412* (2017).
[56] Shichang Zhang, Yozen Liu, Yizhou Sun, and Neil Shah. 2021. Graph-less neural networks: Teaching old mlps new tricks via distillation. *arXiv preprint arXiv:2110.08727* (2021).
[57] Tianping Zhang, Yizhuo Zhang, Wei Cao, Jiang Bian, Xiaohan Yi, Shun Zheng, and Jian Li. 2022. Less is more: Fast multivariate time series forecasting with light sampling-oriented mlp structures. *arXiv preprint arXiv:2207.01186* (2022).
[58] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond efficient transformer for long sequence time-series forecasting. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11106–11115.
[59] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *International conference on machine learning*. PMLR, 27268–27286.

A Theoretical Understanding from Data Augmentation Perspective

In this section, we propose a theoretical viewpoint that TimeDistill unifies the idea of augmenting training data with *blended* data that combines teacher outputs and ground truth. This concept is closely related to *mixup* [55], a well-known data augmentation strategy. *Mixup* creates new training samples by *mixing* two examples' inputs and corresponding targets according to a random interpolation coefficient. Analogously, in multi-scale or multi-period distillation, we create augmented samples that blend teacher predictions and ground truth signals in a manner that provides richer supervision signals. Concretely, in a standard *mixup* procedure, one obtains augmented samples $(\tilde{x}, \tilde{y})$ from two distinct training samples (x_i, y_i) and (x_j, y_j) by drawing a mixing ratio λ and computing:

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \quad \tilde{y} = \lambda y_i + (1-\lambda)y_j.$$

In our distillation setting, instead of mixing two different input samples, we can think of $\hat{\mathbf{Y}}_t$ (the teacher predictions) and $\mathbf{Y}$ (the ground truth) as two sources to be mixed. Hence, these augmented targets become:

$$\tilde{\mathbf{Y}} = \lambda\,\mathbf{Y} \;+\; (1-\lambda)\hat{\mathbf{Y}}_t,$$

while keeping the input as it is. By treating $\hat{\mathbf{Y}}_t$ as another "view" of the target, we are effectively performing a *mixup-style* augmentation in the target space. This approach can be especially valuable in time series forecasting, where incorporating teacher signals from multiple horizons or multiple resolutions may help capture nuanced temporal structures. *Next, we present two theorems that formalize multi-scale and multi-period loss as upper bound of matching mixup augmented prediction and student prediction.*

Theorem A.1. *Let (x, y) denote original input data pairs and (x, y^t) represent corresponding teacher data pairs. Consider a data augmentation function $\mathcal{A}(\cdot)$ applied to (x, y), generating augmented samples (x', y'). Define the training loss on these augmented samples as $\mathcal{L}_{aug} = \sum_{(x',y')\in\mathcal{A}(x,y)} |f_s(x') - y'|^2$. Then, the following inequality holds:*

$$\mathcal{L}_{sup} + \eta\mathcal{L}_{scale} \geq \mathcal{L}_{aug} \tag{13}$$

when $\mathcal{A}(\cdot)$ is instantiated as a mixup function [55] that interpolates between the original input data (x, y) and teacher data (x, y^t) with a mixing coefficient $\lambda = \frac{1}{1+\eta}$, i.e. $y' = \lambda y + (1-\lambda)y^t$.

Proof. Let $\gamma_0 = 1$ and $\gamma_m = \frac{\eta}{M+1}$ for $m = 1, \ldots, M$. Observe that these weights satisfy $\gamma_0 + \sum_{m=1}^{M}\gamma_m = 1 + \eta$. Using these weights, we have:

$$\begin{aligned}
&\mathcal{L}_{sup} + \eta\mathcal{L}_{scale} \\
&= \left(\hat{Y}_s - Y\right)^2 + \eta\sum_{m=0}^{M}\frac{\left(\hat{Y}_s - \hat{Y}_t^{(m)}\right)^2}{M+1} \\
&= \sum_{m=0}^{M}\gamma_m\left(\hat{Y}_s - x_m\right)^2,
\end{aligned}$$

where $x_0 = Y$ and $x_{m+1} = \hat{Y}_t^{(m)}$ for $m = 0, \ldots, M$. By Jensen's inequality [14], for a convex function f, we have:

$$\sum_{m=0}^{M}\gamma_m f(x_m) \geq f\Big(\frac{\sum_{m=0}^{M}\gamma_m x_m}{\sum_{m=0}^{M}\gamma_m}\Big).$$

We use Jensen's inequality, which applies due to the convexity of the squared loss function $f(x) = (\hat{Y}_s - x)^2$. We obtain:

$$\sum_{m=0}^{M}\gamma_m\left(\hat{Y}_s - x_m\right)^2 \geq \left(\hat{Y}_s - \frac{\sum_{m=0}^{M}\gamma_m x_m}{\sum_{m=0}^{M}\gamma_m}\right)^2.$$

Now, compute the weighted sum of x_m using the weights γ_m and we define $\lambda = \frac{1}{1+\eta}$:

$$\begin{aligned}
\frac{\sum_{m=0}^{M}\gamma_m x_m}{\sum_{m=0}^{M}\gamma_m} &= \frac{\gamma_0 x_0 + \sum_{m=1}^{M}\gamma_m x_m}{1+\eta} \\
&= \frac{1}{1+\eta}Y + \frac{\eta}{1+\eta}\sum_{m=0}^{M}\frac{\hat{Y}_t^{(m)}}{M+1} \\
&= \lambda Y + (1-\lambda)\sum_{m=0}^{M}\frac{\hat{Y}_t^{(m)}}{M+1}.
\end{aligned}$$

Substituting this result back, we conclude that:

$$\begin{aligned}
&\mathcal{L}_{sup} + \eta\mathcal{L}_{scale} \\
&= \left(\hat{Y}_s - Y\right)^2 + \eta\sum_{m=0}^{M}\frac{\left(\hat{Y}_s - \hat{Y}_t^{(m)}\right)^2}{M+1} \\
&\geq \Big[\hat{Y}_s - \big[\lambda Y + (1-\lambda)\sum_{m=0}^{M}\frac{\hat{Y}_t^{(m)}}{M+1}\big]\Big]^2 \\
&= \mathcal{L}_{aug}
\end{aligned}$$

This completes the proof. □

Theorem A.2. *Define the training loss on the augmented samples using KL divergence as $\mathcal{L}_{aug} = \sum_{(x',y')\in\mathcal{A}(x,y)} KL\big(y', \mathcal{X}(f_s(x'))\big)$, where $\mathcal{X}(\cdot) = Softmax(FFT(\cdot))$. Then, the following inequality holds:*

$$\mathcal{L}_{sup} + \eta\mathcal{L}_{period} \geq \mathcal{L}_{aug} \tag{14}$$

where $\mathcal{A}(\cdot)$ is instantiated as a mixup function that interpolates between the period distribution of original input data $(x, \mathcal{X}(y))$ and teacher data $(x, \mathcal{X}(y^t))$ with a mixing coefficient $\lambda = \eta$, i.e. $y' = \mathcal{X}(y) + \lambda\mathcal{X}(y^t)$.

Proof. We denote $\mathbf{Q}_y = \mathcal{X}(y)$, $\mathbf{Q}_s = \mathcal{X}(f_s(x))$, $\mathbf{Q}_t = \mathcal{X}(y_t)$, $\hat{Y}_s = f_s(x)$ According to Parseval's theorem, we rewrite $\mathcal{L}_{sup}$ as following:

$$\begin{aligned}
&\mathcal{L}_{sup} + \eta\mathcal{L}_{period} \\
&= \left(\hat{Y}_s - Y\right)^2 + \eta\,\mathrm{KL}\left(\mathbf{Q}_t, \mathbf{Q}_s\right) \\
&= \left(FFT(\hat{Y}_s) - FFT(Y)\right)^2 + \eta\,\mathrm{KL}\left(\mathbf{Q}_t, \mathbf{Q}_s\right)
\end{aligned}$$

When the temperature τ goes to ∞, minimizing KL divergence between two distributions is equivalent to minimizing the MSE error [19]. Thus, we have

$$\begin{aligned}
&\left(FFT(\hat{Y}_s) - FFT(Y)\right)^2 + \eta\,\mathrm{KL}\left(\mathbf{Q}_t, \mathbf{Q}_s\right) \\
&= \lim_{\tau\to\infty} \mathrm{KL}\left(\mathcal{X}(Y), \mathcal{X}(\hat{Y}_s)\right) + \eta\,\mathrm{KL}\left(\mathbf{Q}_t, \mathbf{Q}_s\right) \\
&= \lim_{\tau\to\infty} \mathrm{KL}\left(\mathbf{Q}_y, \mathbf{Q}_s\right) + \eta\,\mathrm{KL}\left(\mathbf{Q}_t, \mathbf{Q}_s\right)
\end{aligned}$$

The KL divergence of P from Q is defined as

$$\mathrm{KL}[P||Q] = \sum_{x\in\mathcal{X}} p(x)\cdot\log\frac{p(x)}{q(x)}$$

and the log sum inequality [36] states that

$$\sum_{i=1}^{n} a_i \log\frac{a_i}{b_i} \geq \left(\sum_{i=1}^{n} a_i\right)\log\frac{\sum_{i=1}^{n} a_i}{\sum_{i=1}^{n} b_i},$$

where $a_1, \ldots, a_n$ and $b_1, \ldots, b_n$ are non-negative real numbers. For clarity, we omit $\lim_{\tau\to\infty}$ in the following derivation. Continuing with the main derivation, we have:

$$\begin{aligned}
&\mathrm{KL}\left(\mathbf{Q}_y, \mathbf{Q}_s\right) + \eta\,\mathrm{KL}\left(\mathbf{Q}_t, \mathbf{Q}_s\right) \\
&= \sum_{x\in\mathcal{X}} \mathbf{Q}_y(x)\cdot\log\frac{\mathbf{Q}_y(x)}{\mathbf{Q}_s(x)} \\
&\qquad + \eta\sum_{x\in\mathcal{X}} \mathbf{Q}_t(x)\cdot\log\frac{\mathbf{Q}_t(x)}{\mathbf{Q}_s(x)} \\
&= \sum_{x\in\mathcal{X}} [\mathbf{Q}_y(x)\cdot\log\frac{\mathbf{Q}_y(x)}{\mathbf{Q}_s(x)} \\
&\qquad + \eta\mathbf{Q}_t(x)\cdot\log\frac{\eta\mathbf{Q}_t(x)}{\eta\mathbf{Q}_s(x)}] \\
&\geq \sum_{x\in\mathcal{X}} [[\mathbf{Q}_y(x) + \eta\mathbf{Q}_t(x)] \\
&\qquad \cdot\log\frac{\mathbf{Q}_y(x) + \eta\mathbf{Q}_t(x)}{\mathbf{Q}_s(x) + \eta\mathbf{Q}_s(x)}] \\
&= \mathrm{KL}\Big(\mathbf{Q}_y + \eta\,\mathbf{Q}_t\ ,\ \mathbf{Q}_s\Big)
\end{aligned}$$

To align with the mixup formulation defined in our theorem. We denote $\lambda = \eta$, then we have

$$\begin{aligned}
&\mathrm{KL}\Big(\mathbf{Q}_y + \eta\,\mathbf{Q}_t\ ,\ \mathbf{Q}_s\Big) \\
&= \mathrm{KL}\Big(\mathbf{Q}_y + \lambda\,\mathbf{Q}_t\ ,\ \mathbf{Q}_s\Big) \\
&= \mathrm{KL}\Big(\mathcal{X}(Y) + \lambda\,\mathcal{X}(\hat{Y}_t)\ ,\ \mathcal{X}(\hat{Y}_s)\Big) \\
&= \mathcal{L}_{aug}
\end{aligned}$$

This completes the proof. □

These two theorems show that the disllation process provides additional augmented samples that smooth the ground truth and the teacher prediction. This is analogous to label smoothing in classification, where labels are adjusted to be less extreme or certain.

B Implementation Details

Datasets Details. We conduct experiments to evaluate the performance and efficiency of TimeDistill on eight widely used benchmarks: Electricity (ECL), the ETT datasets (ETTh1, ETTh2, ETTm1, ETTm2), Solar, Traffic, and Weather, following the pipeline in [15, 22, 26, 33, 34, 42, 43]. Detailed descriptions of these datasets are provided in Table 6.

Metric Details. We use Mean Square Error (MSE) and Mean Absolute Error (MAE) as our evaluation metrics, following [8, 22, 24–26, 28, 37, 43, 47, 48, 54, 58]:

$$\mathrm{MSE} = \frac{1}{S\times C}\sum_{i=1}^{S}\sum_{j=1}^{C}(Y_{ij} - \hat{Y}_{ij})^2, \tag{15}$$

$$\mathrm{MAE} = \frac{1}{S\times C}\sum_{i=1}^{S}\sum_{j=1}^{C}|Y_{ij} - \hat{Y}_{ij}|. \tag{16}$$

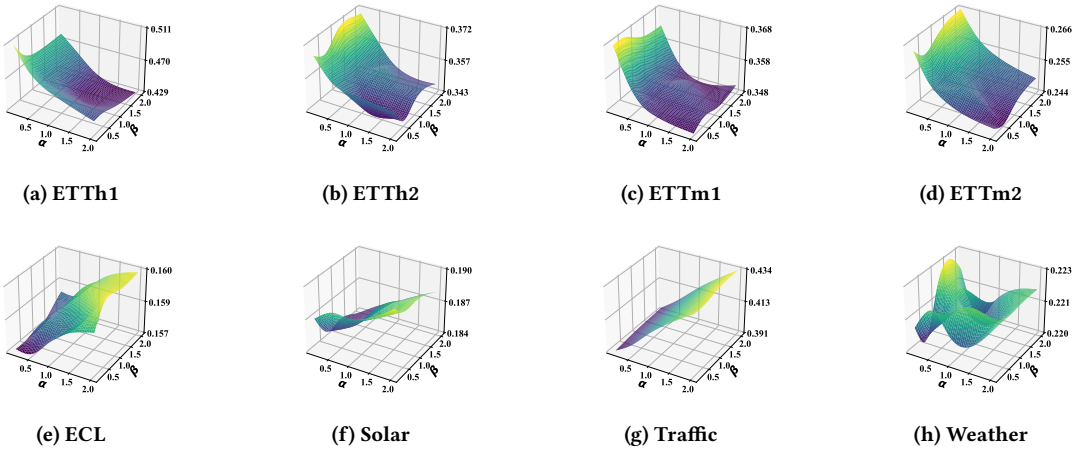
Here, $Y \in \mathbb{R}^{S\times C}$ represents the ground truth, and $\hat{Y} \in \mathbb{R}^{S\times C}$ represents the predictions. S denotes the future prediction length, C is the number of channels, and Y_{ij} indicates the value at the i-th future time point for the j-th channel.

Experiment Details. All experiments are implemented in PyTorch [31] and conducted on a cluster equipped with NVIDIA A100, V100, and K80 GPUs. The teacher models are trained using their default configurations as reported in their respective papers. For the student MLP model, we employ a combination of a decomposition scheme [48, 54, 59] and a two-layer MLP with hidden dimension D as the architecture. When using TimeDistill for distillation, the teacher model is frozen, and only the student MLP is trained. The initial learning rate is set to 0.01, and the ADAM optimizer [20] is used with MSE loss for the training of the student model. We apply early stopping with a patience value of 5 epochs. The batch size is set to 32. The temperature for multi-period distillation is set to $\tau = 0.5$. The number of scales is set to $M = 3$. We use two types of normalization methods, i.e. non-stationary [23] and revin [18]. We perform a hyperparameter search for α and β within the range {0.1, 0.5, 1, 2}. We conduct hyperparameter sensitivity analysis in Appendix C. For consistency, TimeDistill defaults to using ModernTCN as the teacher and fixes the look-back window length to 720. Additional experiments with other teacher models and different look-back window lengths are presented in Section 5.3. Additional detailed model configuration information is presented in Table 7.

Channel Independent MLP. We denote the model as f. The channel-independent strategy predicts each channel independently, defined as:$\hat{Y} = [f(x^1), \ldots, f(x^C)]$, where $\hat{Y} = [\hat{y}^1, \ldots, \hat{y}^C]$ denotes the model prediction. In contrast, the channel-dependent strategy considers the combination of all channels simultaneously, defined as: $\hat{Y} = f(x^1, \ldots, x^C)$. In this paper, for MLP, we adopt the channel-independent strategy because it eliminates the need to model inter-channel relationships [12], which reduces model complexity and

Table 6: Dataset characteristics for time series long-term forecasting task.

Dataset	Dim	Series Length	Dataset Size	Frequency	Forecastability	Information
ECL	321	{96, 192, 336, 720}	(18317, 2633, 5261)	Hourly	0.77	Electricity
ETTh1	7	{96, 192, 336, 720}	(8545, 2881, 2881)	15min	0.38	Temperature
ETTh2	7	{96, 192, 336, 720}	(8545, 2881, 2881)	15min	0.45	Temperature
ETTm1	7	{96, 192, 336, 720}	(34465, 11521, 11521)	15min	0.46	Temperature
ETTm2	7	{96, 192, 336, 720}	(34465, 11521, 11521)	15min	0.55	Temperature
Solar	137	{96, 192, 336, 720}	(36601, 5161, 10417)	10min	0.33	Electricity
Traffic	862	{96, 192, 336, 720}	(12185, 1757, 3509)	Hourly	0.68	Transportation
Weather	21	{96, 192, 336, 720}	(36792, 5271, 10540)	10min	0.75	Weather



(a) ETTh1 **(b) ETTh2** **(c) ETTm1** **(d) ETTm2**

(e) ECL **(f) Solar** **(g) Traffic** **(h) Weather**

Figure 10: Sensitivity Analysis Result of Predcition Level α and Feature Level β Distillation on Different Datasets.

Table 7: Experiment configuration of TimeDistill.

Dataset	Model Hyper-parameter		Training Process		
	D	Norm	Epoch	α	β
ECL	512	non-stationary	20	0.1	0.5
ETTh1	512	non-stationary	20	2	2
ETTh2	512	non-stationary	20	2	0.5
ETTm1	512	non-stationary	20	2	0.1
ETTm2	512	non-stationary	20	2	0.5
Solar	512	non-stationary	20	0.1	2
Traffic	1024	revin	10	0.1	0.1
Weather	512	non-stationary	20	0.5	2

results in a more lightweight model, which is especially advantageous when dealing with a large number of variables.

C Hyperparameter Sensitivity Analysis

In this subsection, we analyze the robustness of TimeDistill by investigating the sensitivity of two key hyperparameters, α and β, in the final objective function of TimeDistill. The parameters α and β regulate the contributions of the prediction-level and feature-level distillation loss terms, respectively. To assess their effects, we vary both α and β over the set $\{0.1, 0.5, 1, 2\}$ while keeping other hyperparameters fixed. We conduct experiments on all 8 datasets. Figure 10 illustrates that TimeDistill generally performs better when a smaller β, such as 0.1 or 0.5, is selected. For α, the results show dataset-specific preferences. On the four ETT datasets, a smaller α usually leads to inferior performance, while increasing α improves the results. This is because α controls the contribution of the loss term at the prediction level. MLP performs poorly on ETT datasets, leading to a larger gap between MLP and the teacher. Matching predictions directly with a larger α helps reduce this performance gap, resulting in better outcomes on ETT datasets. However, for other datasets such as ECL, Solar, Traffic, and Weather, the performance gap between MLP and the teacher is not as significant, or MLP even outperforms the teacher. In such cases, a large α introduces excessive noise to MLP, negatively impacting the performance of TimeDistill. Consequently, on these datasets, a smaller α is more beneficial.